

OEM Compliance Mapping System

How a vendor datasheet becomes a scored, evidence-backed RFP answer

This note documents the system found in the project bundle: a tool that ingests OEM vendor datasheets, extracts structured requirements from an RFP, and produces a scored compliance report mapping each requirement to evidence found in the vendor documents. It is a local-first, single-machine system — no cloud infrastructure, no managed database, everything runs on one box talking to a local Ollama server.

Type RFP → product compliance engine	Interface Streamlit web app + CLI	Storage ChromaDB, on disk	Inference Self-hosted Ollama (LAN)
---	---	-------------------------------------	---

On this page

- 1. What the system does
- 2. End-to-end pipeline (datasheets → RFP → report)
- 3. Module map
- 4. Tech stack
- 5. Data model & storage
- 6. Design choices worth knowing about

At a glance

- 01** Ingest vendor datasheets — extract, identify models, chunk, embed into ChromaDB.
- 02** Extract RFP requirements — regex pass + fast LLM pass, dedup, embed into a per-RFP collection.
- 03** Match & score — retrieve evidence, judge each pair, rank products, write the report.

1 — What the system does

The project is called the **OEM Compliance Mapping System**. It exists to automate a task that's normally done by hand during proposal writing: reading an RFP, pulling out every individual requirement, and checking each one against vendor datasheets to see which product actually satisfies it.

It is built around two knowledge bases that share the same embedding space. One is a library of **OEM datasheets** (firewalls, switches, etc. from vendors like Fortinet, Palo Alto, Cisco, Juniper) that gets built up once and reused across RFPs. The other is built fresh per RFP: the requirements extracted from that specific document. The compliance engine then matches one against the other and scores each candidate product.

Three things it produces

- A searchable vector knowledge base of every vendor spec sheet ever ingested.
- A structured, de-duplicated list of requirements pulled out of an RFP PDF, tagged mandatory / optional.
- A ranked compliance report (Markdown + JSON) per product, with page-level evidence citations for every requirement.

2 — End-to-end pipeline

There are two pipelines that feed into one engine. The first runs once per datasheet; the second runs once per RFP. Both produce embeddings in the same vector space so the third stage can compare them directly.

A. Datasheet ingestion (build the product knowledge base)

Stage	Module	What happens
Extract	pdf_extractor.py	Pulls text + tables per page via pdfplumber/PyMuPDF. Pages with no real text layer (scanned spec sheets) fall through to Tesseract or PaddleOCR.
Identify	model_identifier.py	Finds every product model number on the page (regex + section heuristics), splits multi-model comparison tables into per-model spec dicts.
Chunk	chunker.py	Turns each model's specs into purpose-built chunks — spec table, features, ordering info, description — sized 800–1200 chars so a single spec row never gets buried.
Embed + store	vector_store.py	Each chunk is embedded (bge-m3 via Ollama) and upserted into a Chroma collection, tagged with vendor / model / chunk type / source page.

B. RFP requirement extraction (per submission)

Stage	Module	What happens
Read	rfp_extractor.py	User selects a page range in the UI; pages are pulled and split into ~2,500-char chunks.

Stage	Module	What happens
Extract	rfp_extractor.py	Two passes: a regex pass catches obvious numeric thresholds for free, then an LLM pass (fast, non-reasoning model) catches everything else as structured JSON.
Dedup + store	rfp_extractor.py	Requirements get IDs, get tagged mandatory/optional, are written to JSON, and embedded into their own per-RFP Chroma collection.

C. Compliance scoring (the matching engine)

Stage	Module	What happens
Retrieve	retrieval.py	For each requirement, semantic search returns the top OEM chunks, grouped by (vendor, model).
Judge	matcher.py	Each (requirement, product) pair gets a verdict — Full / Partial / No Match. Cheap checks (no evidence, numeric regex, high cosine score) short-circuit before any LLM call is made.
Rank	ranker.py	Per-product scores are weighted 2x for mandatory requirements, run in parallel across products with a thread pool.
Report	reporter.py	Writes a ranked Markdown report (exec summary, compliance matrix, gap analysis, cited evidence) plus the raw JSON.

The matcher is deliberately stingy with LLM calls — it's tuned for a CPU-only inference box, not a GPU cluster. Roughly a fifth to a quarter of requirements (plain numeric thresholds) never touch the LLM at all.

3 — Module map

The codebase is organised by responsibility, with a thin public API and UI layer sitting on top of four backend packages.

Folder / file	Role
app.py	Streamlit UI — three tabs: Upload RFP, Manage Database, Reports.
main.py	CLI — ingest / search / stats / delete / list-docs commands.
api.py	OEMKnowledgeBase — the clean Python entry point other modules import.
ingestion/	PDF extraction, model identification, chunking, OCR fallback.
rfp/	RFP page-range reading, requirement extraction (regex + LLM).
compliance/	Retrieval → matching → ranking → report generation (the scoring engine).
knowledge_base/	VectorStoreManager — the ChromaDB wrapper used by both KBs.
services/	Thin clients: embedding_service.py (Ollama embed API), llm_services.py (OpenAI-compatible chat client).

models/ , compliance/schemas.py	Pydantic data contracts shared across every stage.
config/settings.py	All tunables in one dataclass tree — chunk sizes, model names, endpoints, thresholds.
ui/kb_inventory.py	Read-only helper that builds the “Manage Database” view from chunk metadata.

Worth noting: **section_rules.py** exists purely to fix a real bug — the logic for “is this a family-level section vs. a per-model one” used to be duplicated in both *chunker.py* and *model_identifier.py*, and the two copies drifted. It’s now a single shared module both files import from.

4 — Tech stack

Core stack

Layer	Technology	Why this one
Language	Python 3.10 / 3.14	Repo ships compiled bytecode for both — looks like dev happens on 3.14 locally, deployed on 3.10.
UI	Streamlit	Three-tab app for upload, KB management, and report browsing. No separate frontend build.
CLI	argparse	Full feature parity with the UI for scripted / headless ingestion.
Data models	Pydantic v2	Typed schemas for every object passed between stages — chunks, requirements, evidence, reports.
Config	dataclasses + python-dotenv	One PipelineConfig tree; .env overrides things like the LLM API key.
Logging	loguru / stdlib logging	loguru in the ingestion path, stdlib logging in the compliance engine.

Document processing

Layer	Technology	Why this one
PDF text	pdfplumber, PyMuPDF (fitz)	Used together — pdfplumber for layout-aware text and tables, PyMuPDF as a faster fallback / cross-check.
OCR	Tesseract, PaddleOCR (optional)	Triggered only when a page has no real text layer (checked via pdffonts / PyMuPDF font probe).
Image handling	Pillow, OpenCV (optional)	OpenCV does OCR preprocessing when available; the pipeline degrades gracefully without it.

AI / retrieval

Layer	Technology	Why this one
Embeddings	bge-m3 (via Ollama)	Same model embeds both datasheet chunks and RFP requirements, so cosine similarity between them is meaningful.
Vector store	ChromaDB (persistent, local)	One collection per OEM KB, one sub-collection per RFP. No external vector DB service.
LLM runtime	Ollama, OpenAI-compatible client	Self-hosted on the LAN; the app talks to it through the openai Python SDK pointed at a local base_url.
Model routing	gemma2:9b (configurable)	Reasoning model for compliance verdicts and report-quality answers; a separate “fast” slot is reserved for pure-JSON extraction work.

A note on model routing

The LLM client deliberately exposes two call paths. **generate()** goes to the reasoning model and is reserved for compliance verdicts, where the extra deliberation pays for itself. **generate_fast()** goes to a separate non-reasoning instruct model (or suppresses chain-of-thought on the same model) for RFP requirement extraction, where the only thing that matters is clean JSON out, fast. On a CPU-only inference box this is the difference between minutes and seconds per call.

```
class LLMService:
    def generate(self, prompt):      # reasoning model
        ... # compliance verdicts, report text

    def generate_fast(self, prompt): # non-reasoning model
        ... # RFP requirement extraction (pure JSON)
```

5 — Data model & storage

Everything lives on local disk under a single data/ directory — there's no external database server to provision.

```
data/
  raw/           source PDFs (datasheets), mirrored on ingest
  processed/     intermediate extraction artifacts
  vector_store/  ChromaDB persistent store
    |-- oem_datasheets      (shared product KB)
    `-- rfp_requirements_*  (one collection per RFP)
  requirements/   extracted RFP requirements, as JSON
  compliance_reports/ generated .md + .json reports
  logs/          loguru output
```

Key schema objects

Object	Carries
DocumentChunk	text, vendor, model_name, chunk_type, source page, embedding
ModelSpec	per-product spec dict, derived from one or more comparison tables
Requirement	requirement text, mandatory flag, source page, generated ID
EvidenceChunk	the KB chunk that backs a verdict, with similarity score
RequirementResult	verdict (Full / Partial / No / Not Found / ...) + evidence
ProductScore	per-product rollup: mandatory score, overall score, rank

Documents are keyed by a 16-character SHA-256 prefix of the file content, so re-ingesting the same PDF is a no-op unless --force is passed.

6 — Design choices worth knowing about

Everything runs local. No managed vector DB, no cloud LLM API by default. ChromaDB persists to disk and Ollama serves both embeddings and chat completions from a box on the LAN (100.98.219.69). This keeps vendor datasheets and RFP content off third-party infrastructure.

Two-model split, not one. A reasoning model and a fast extraction model are configured separately. The system was clearly tuned against real latency pain — the comments in `llm_services.py` and `rfp_extractor.py` both describe cutting multi-minute calls down to seconds by avoiding chain-of-thought on the extraction path.

Cheap checks before expensive ones. The matcher tries to resolve a verdict with zero LLM calls first: no evidence at all → instant 'Not Found'; a plain numeric threshold → resolved by regex; a very high cosine match with no unit to check → instant 'Full Match'. The LLM is the last resort, not the first.

Shared embedding space. Datasheet chunks and RFP requirements are embedded with the same bge-m3 model specifically so that cosine similarity between the two is meaningful — this is what makes semantic retrieval across the two collections work at all.

One source of truth for fragile logic. `section_rules.py` was pulled out after a bug where two files had silently drifted copies of the same classification logic. It's a small thing, but it's a sign the codebase has been through at least one real debugging cycle, not just written once and shipped.

This document was assembled directly from the project source — docstrings, inline comments, and config defaults — rather than from external documentation.